

COL764

Assignment 3 Report

Team 4
Aditya Saxena
Sanyam Garg

October 2025

Abstract

This report details the implementation and evaluation of information retrieval pipelines for Assignment 3, focusing on the challenging TREC DL Hard dataset. In Task 1, we investigate the "retrieve-and-rerank" framework using three BERT-family cross-encoders: TinyBERT-L6, BERT-base, and BERT-large. In Task 2, we evaluate four alternative architectures: MiniLM-L6, MiniLM-L12, ELECTRA-base, and monoT5-base. We provide separate results and analyses for these tasks. In Task 3, we explored several techniques to improve upon the best baseline model (ELECTRA), including Contextualized Embedding Query Expansion (CEQE), pairwise ranking models (DuoT5/DuoBERT concepts), and multi-stage reranking pipelines (BM25 -> MiniLM-L12 -> Top N -> ELECTRA). We present the implementation and results for the multi-stage reranking approach, which yielded the most promising performance improvements. We compare performance based on retrieval metrics (NDCG@1/5/10, MRR@10), computational cost, and the impact of initial reranking depth (k). Data is presented exactly as recorded in our experimental runs.

1 Introduction and Common Methodology

Modern information retrieval systems often face a trade-off between the speed required to search vast corpora and the accuracy needed to satisfy user intent. The "retrieve-and-rerank" pipeline addresses this by employing a two-stage process:

1. **Stage 1: Retrieval.** A fast, typically lexical, retrieval algorithm (e.g., BM25) is used to efficiently retrieve a broad set of k potentially relevant candidate documents from the full index.
2. **Stage 2: Reranking.** A more powerful, computationally intensive semantic model (e.g., a transformer-based cross-encoder) is applied only to the top k candidates to re-score and re-order them with higher precision.

For all experiments in this report, the first stage utilized the Okapi BM25 algorithm implemented in `pyserini`, querying the pre-built MS MARCO passage index (`msmarco-passage`). The second stage involved different transformer models loaded via the Hugging Face `transformers` library. We evaluated each model's reranking performance by varying the initial retrieval depth k and measuring standard IR metrics (NDCG@1, NDCG@5, NDCG@10, MRR@10) on the final top-10 list, along with the execution time on our local machine (CPU). All models were used "off-the-shelf" without further fine-tuning, as required.

2 Task 1: BERT-Family Rerankers

2.1 Methodology

Task 1 required implementing the retrieve-and-rerank pipeline using a "BERT" model. To explore the impact of model size within this family, we selected three cross-encoder models fine-tuned on the MS MARCO dataset:

- **TinyBERT-L6:** cross-encoder/ms-marco-TinyBERT-L6. A distilled model with 6 layers.
- **BERT-base:** nboost/pt-bert-base-uncased-msmarco. A standard 12-layer BERT model.
- **BERT-large:** nboost/pt-bert-large-msmarco. A larger 24-layer BERT model.

These models take a '(query, document)' pair as input and output a single relevance score.

2.2 Results

The performance metrics and execution times for the BERT-family models across different values of k are presented in Tables 1, 2, and 3.

Table 1: Task 1 Results: cross-encoder/ms-marco-TinyBERT-L6 vs. k

k	Time (sec)	NDCG@1	NDCG@5	NDCG@10	MRR@10
10	31.89	0.37143	0.31092	0.28987	0.57067
20	21.07	0.38286	0.31589	0.30357	0.56575
40	39.00	0.41714	0.35489	0.33422	0.60852
80	83.74	0.41714	0.37366	0.35536	0.62440
120	126.43	0.41714	0.36252	0.36011	0.62874
160	177.43	0.38857	0.36022	0.35851	0.60507
200	226.32	0.40857	0.37144	0.37060	0.62244

Table 2: Task 1 Results: nboost/pt-bert-base-uncased-msmarco vs. k

k	Time (sec)	NDCG@1	NDCG@5	NDCG@10	MRR@10
10	22.57	0.42857	0.32183	0.29929	0.61111
20	40.18	0.44857	0.33689	0.31712	0.60783
40	75.29	0.48857	0.38009	0.35206	0.64767
60	121.24	0.47143	0.37681	0.35630	0.63317
80	164.45	0.47429	0.37725	0.37005	0.65036
100	211.35	0.45429	0.37990	0.37559	0.63913
120	257.36	0.45429	0.37859	0.37972	0.63506
140	302.44	0.45429	0.37692	0.37756	0.63358
160	351.46	0.45429	0.37826	0.37871	0.63191
180	406.49	0.45429	0.38000	0.38002	0.63046
200	459.79	0.45429	0.38012	0.38038	0.63000
240	566.08	0.45429	0.37851	0.38003	0.62744

Table 3: Task 1 Results: nboost/pt-bert-large-msmarco vs. k

k	Time (sec)	NDCG@1	NDCG@5	NDCG@10	MRR@10
10	56.23	0.42286	0.33028	0.30168	0.62333
20	112.65	0.41238	0.33767	0.31695	0.60556
40	231.78	0.41238	0.36839	0.35050	0.62667
60	381.42	0.39524	0.37743	0.36441	0.62408
80	506.15	0.39524	0.39030	0.37418	0.63022
100	648.06	0.39524	0.39108	0.38321	0.63175
120	780.22	0.39524	0.39271	0.38767	0.63350
140	932.71	0.39524	0.38824	0.38679	0.63351

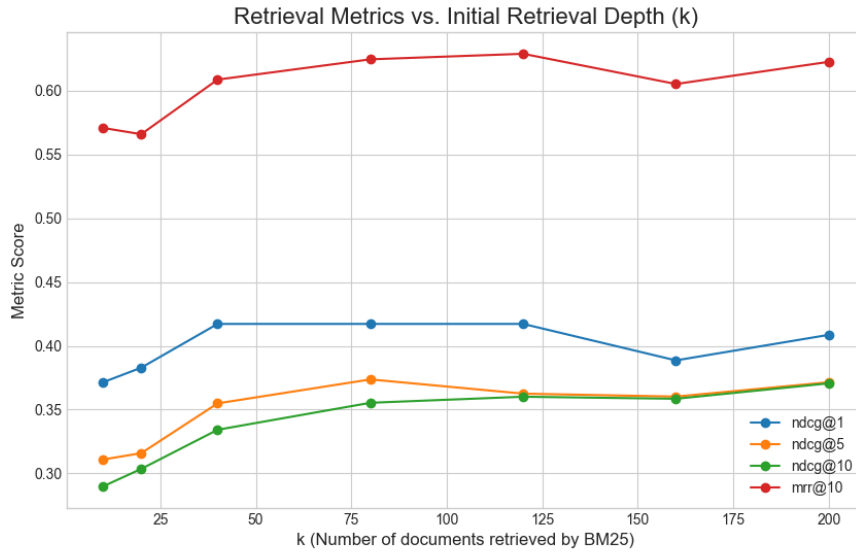


Figure 1: Metrics for BERT-tiny

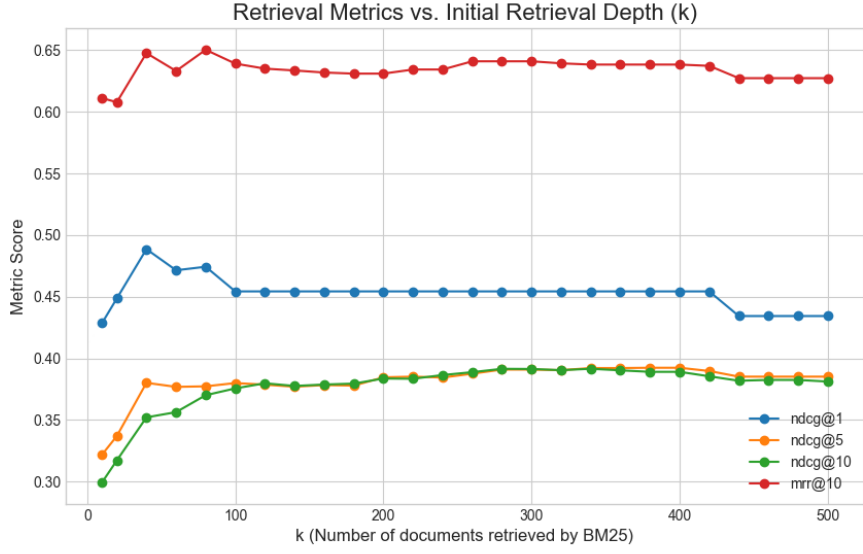


Figure 2: Metrics for BERT-base

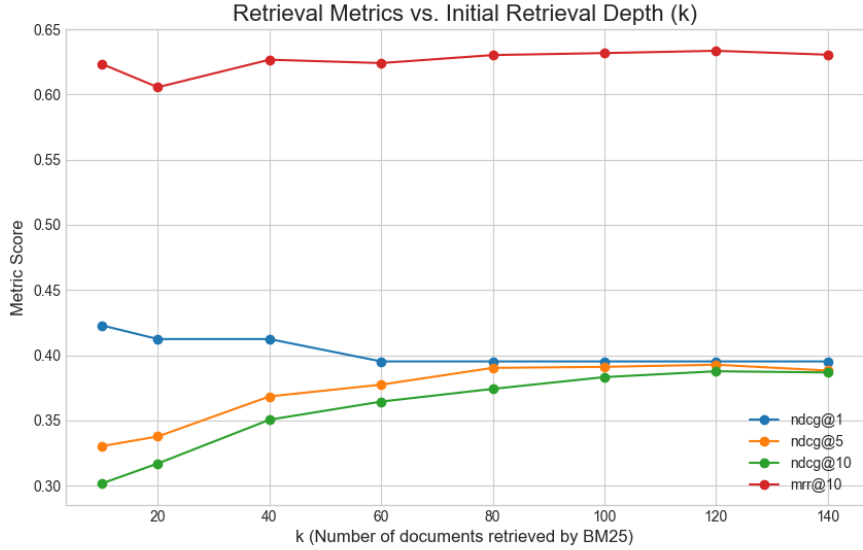


Figure 3: Metrics for BERT-LARGE

2.3 Task 1 Analysis and Observations

Effect of k : All three BERT models demonstrate the expected pattern: performance generally increases with k before plateauing. The extended results for BERT-base (Table 2) clearly show this plateau, with NDCG@10 stabilizing around 0.380 for $k \geq 120$. This saturation occurs because, beyond a certain point, the initial BM25 retrieval likely includes most of the truly relevant documents within the top k . Further increasing k primarily adds less relevant or irrelevant documents, which the reranker correctly scores low, leading to minimal changes in the final top-10 ranking.

Model Size vs. Performance: BERT-base provides the best overall performance in this family, particularly for NDCG@1 (peak 0.489 at $k = 40$) and MRR@10 (peak 0.650 at $k = 80$). This suggests it is highly effective at identifying the single most relevant document. BERT-large,

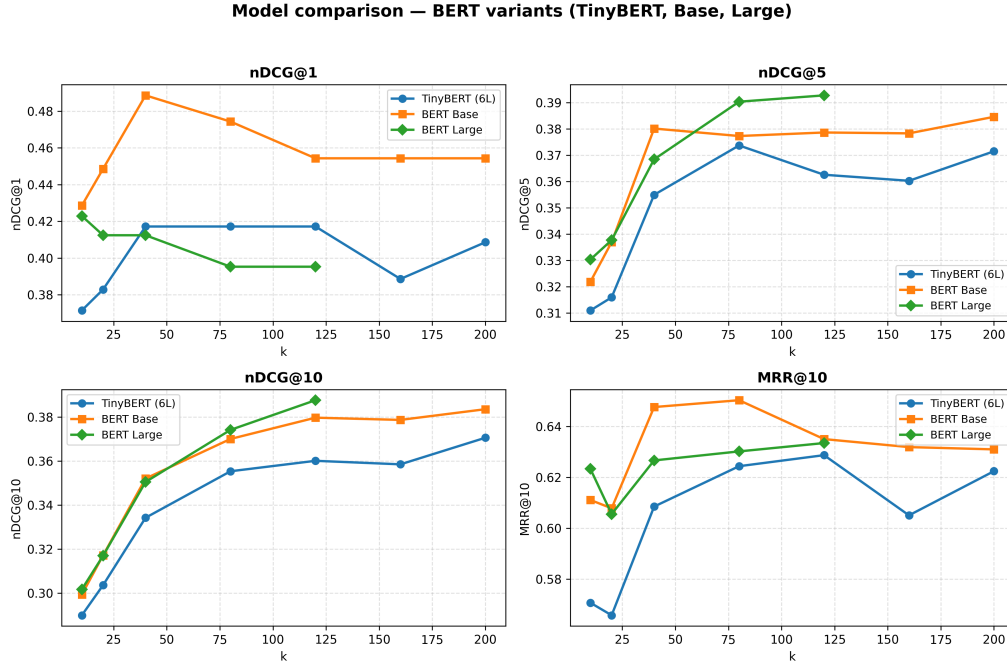


Figure 4: Comparison across bert models

despite its larger size and significantly longer processing time, underperforms BERT-base on NDCG@1 and MRR@10. Its peak NDCG@5 (0.393) and NDCG@10 (0.388) are only marginally better than BERT-base (0.380 each) and come at roughly 3x the computational cost. TinyBERT-L6 is the fastest and performs commendably for its size, nearly matching BERT-large at low k values, demonstrating the effectiveness of knowledge distillation.

NDCG@1 Trend: A notable observation is the dip or plateau in NDCG@1 for all models at higher k values after an initial peak (e.g., BERT-base peaks at $k = 40$). This suggests that while processing more candidates helps find more relevant documents overall (improving NDCG@5/10), it might introduce ambiguity or similar-scoring distractors that make it slightly harder to pinpoint the single absolute best document with perfect confidence.

Time Cost: Runtime scales roughly linearly with k . BERT-large is substantially slower than BERT-base, which is slower than TinyBERT, directly reflecting their parameter counts.

3 Task 2: Alternative Reranking Architectures

3.1 Methodology

Task 2 required evaluating alternatives to the standard BERT cross-encoder approach. We implemented and tested four models representing different architectures and pre-training strategies:

- **MiniLM-L6:** cross-encoder/ms-marco-MiniLM-L6-v2. A 6-layer distilled cross-encoder.
- **MiniLM-L12:** cross-encoder/ms-marco-MiniLM-L12-v2. A 12-layer distilled cross-encoder.
- **ELECTRA:** cross-encoder/ms-marco-electra-base. A base-sized cross-encoder using the ELECTRA pre-training objective (Replaced Token Detection).
- **monoT5:** castorini/monot5-base-msmarco-10k. A T5 sequence-to-sequence model adapted for reranking via generation of 'true'/'false' tokens.

3.2 Results

Performance metrics and execution times for the Task 2 models are presented in Tables 4, 5, 6, and 7.

Table 4: Task 2 Results: cross-encoder/ms-marco-MiniLM-L6-v2 vs. k

k	Time (sec)	NDCG@1	NDCG@5	NDCG@10	MRR@10
10	17.98	0.40286	0.33427	0.29864	0.60738
20	11.79	0.36667	0.33058	0.31598	0.58136
40	19.07	0.38667	0.36105	0.35016	0.61252
80	34.82	0.39810	0.37163	0.36765	0.62325
120	55.43	0.38667	0.37258	0.38358	0.62566
160	73.06	0.38667	0.37181	0.38486	0.62566
200	89.87	0.38667	0.37852	0.39064	0.62663

Table 5: Task 2 Results: cross-encoder/ms-marco-MiniLM-L12-v2 vs. k

k	Time (sec)	NDCG@1	NDCG@5	NDCG@10	MRR@10
10	24.94	0.40000	0.33146	0.29754	0.58952
20	18.37	0.38952	0.33339	0.31946	0.57767
40	33.25	0.40952	0.36485	0.35299	0.60817
80	63.55	0.40952	0.37364	0.37345	0.62758
120	101.46	0.39810	0.37350	0.38645	0.62567
160	138.63	0.39810	0.37179	0.38537	0.62452
200	175.57	0.39810	0.37701	0.39406	0.62389

Table 6: Task 2 Results: cross-encoder/ms-marco-electra-base vs. k

k	Time (sec)	NDCG@1	NDCG@5	NDCG@10	MRR@10
10	24.63	0.39429	0.32447	0.29583	0.59138
20	46.62	0.41714	0.32680	0.31360	0.59019
40	87.47	0.47714	0.37392	0.35156	0.66252
80	194.13	0.47429	0.39540	0.37736	0.67586
120	302.69	0.51429	0.40767	0.40028	0.71205
160	424.13	0.51429	0.40613	0.40056	0.71105
200	546.46	0.51429	0.41218	0.40718	0.71105
240	689.98	0.49429	0.40607	0.40314	0.69678

Table 7: Task 2 Results: castorini/monot5-base-msmarco-10k vs. k

k	Time (sec)	NDCG@1	NDCG@5	NDCG@10	MRR@10
10	27.35	0.38286	0.33052	0.29779	0.57483
20	49.82	0.36381	0.33386	0.31489	0.57422
40	98.24	0.34952	0.35307	0.33784	0.58528
80	215.02	0.34952	0.35468	0.35726	0.59738
120	337.23	0.36952	0.35668	0.36102	0.60725
160	455.73	0.35810	0.35453	0.35914	0.60411
200	608.82	0.35810	0.36184	0.36401	0.60144

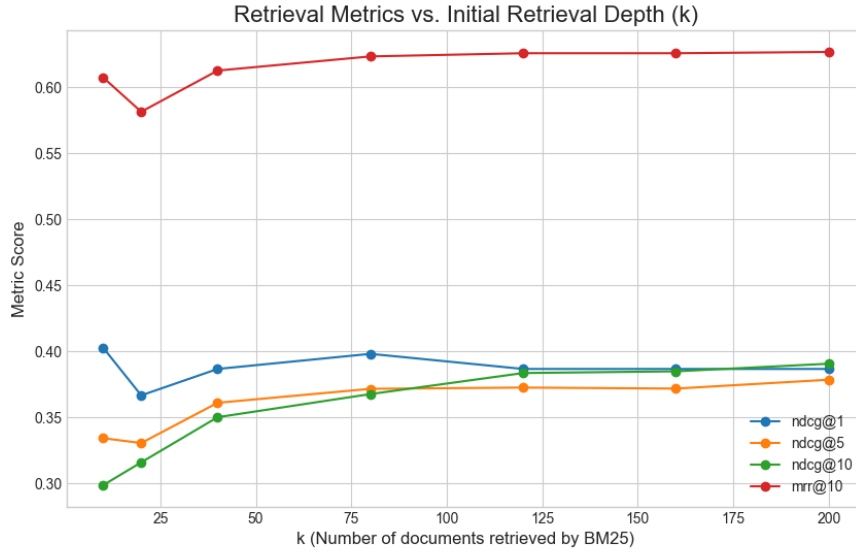


Figure 5: Metrics for MiniLM-L6

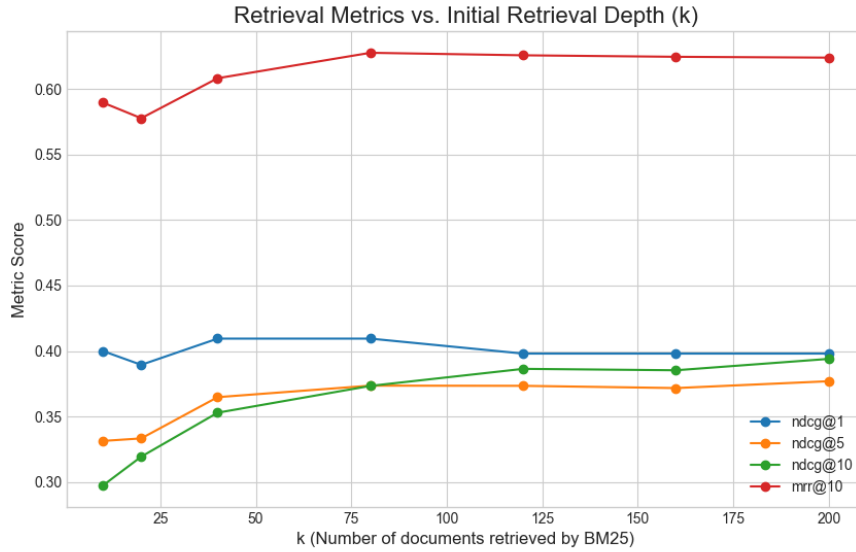


Figure 6: Metrics for MiniLM-L12

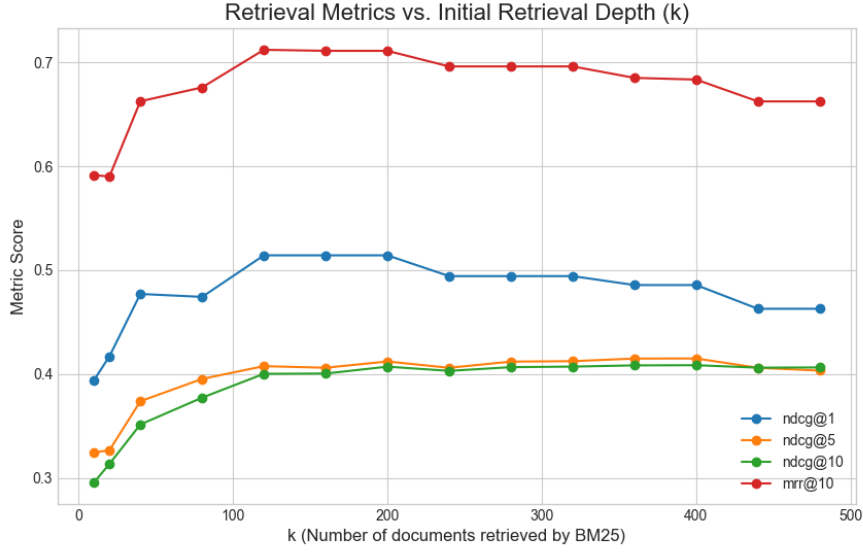


Figure 7: Metrics for ELECTRA

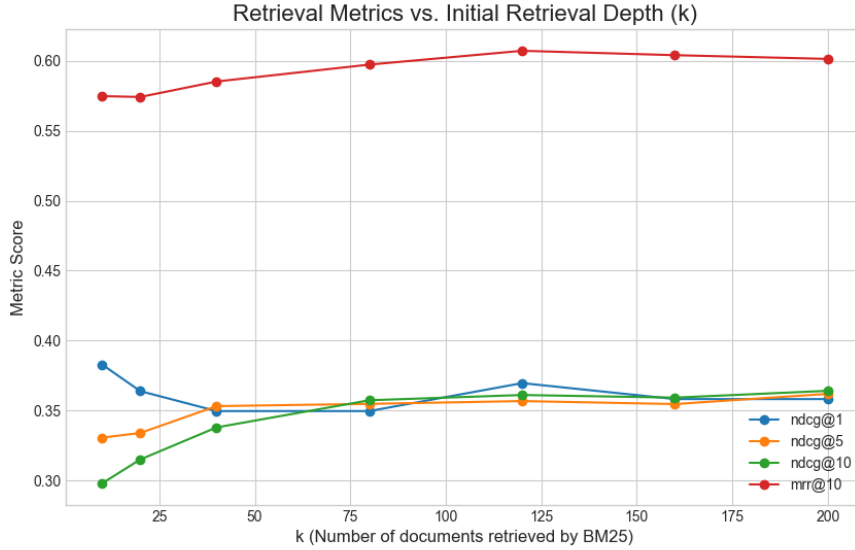


Figure 8: Metrics for MONOT5-10K

3.3 Task 2 Analysis and Observations

Effect of k : The trend of improvement with increasing k followed by a plateau is consistent across MiniLM and ELECTRA, similar to the Task 1 models. ELECTRA shows the most pronounced gains up to $k = 120$. monoT5 exhibits weaker correlation with k , suggesting the sequence-to-sequence scoring might be less sensitive to the size of the candidate pool or perhaps struggles more with distinguishing relevance among a larger set of candidates.

Model Comparisons:

- **ELECTRA:** Clearly the strongest performer in this group, achieving the highest scores across all metrics. Its peak NDCG@1 (0.514) and MRR@10 (0.712) are particularly noteworthy.
- **MiniLM:** Both L6 and L12 variants perform very well, significantly outperforming monoT5

and TinyBERT. MiniLM-L12 shows slightly better peak NDCG@10 (0.394 vs 0.391) than L6, aligning with its larger size, but the difference is small. They offer excellent efficiency, especially L6.

- **monoT5:** This model remains the weakest performer among the fine-tuned models used in Task 1 and 2, confirming the observations from the overall analysis.

Time Cost: MiniLM-L6 is the fastest model tested. MiniLM-L12 is next, roughly twice as slow as L6. ELECTRA’s runtime is comparable to BERT-base. monoT5 is relatively slow, similar to BERT-large.

4 Overall Comparative Analysis (Tasks 1 & 2)

4.1 Performance Summary

Table 8 consolidates the peak performance observed for all models tested across Tasks 1 and 2, providing a unified view for comparison. Note that “Approx. Optimal k ” and “Peak Time” refer to the range or value where the best overall metrics were observed, primarily focusing on NDCG@10 and MRR@10.

Table 8: Overall Peak Performance vs. Time (All Models)

Model	Approx. Optimal k	NDCG@1	NDCG@5	NDCG@10	MRR@10	Peak Time (s)
ELECTRA-base	120-200	0.514	0.412	0.407	0.712	≈546.5
BERT-base	120-200	0.489	0.380	0.380	0.650	≈459.8
BERT-large	120-140	0.423	0.393	0.388	0.634	≈932.7
MiniLM-L12	120-200	0.410	0.377	0.394	0.628	≈175.6
MiniLM-L6	120-200	0.403	0.379	0.391	0.627	≈89.9
TinyBERT-L6	80-120	0.417	0.374	0.371	0.629	≈126.4
monoT5-10k	≈120	0.383	0.362	0.364	0.607	≈608.8

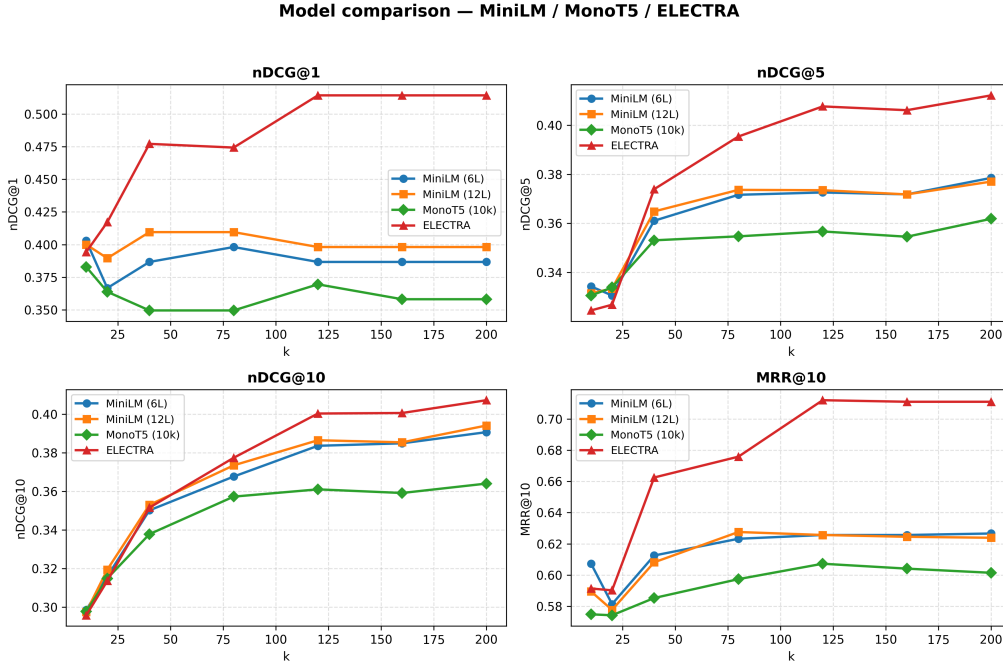


Figure 9: Task2 Comparison

4.2 Observations and Intuition (TREC DL Hard Context)

ELECTRA's Dominance on Hard Queries: ELECTRA's superior performance, particularly its high NDCG@1 and MRR@10, is significant for the TREC DL Hard dataset. These queries often require understanding complex information needs, identifying specific entities or relationships, and distinguishing subtle differences in meaning, going beyond simple keyword matching. ELECTRA's Replaced Token Detection (RTD) pre-training objective forces the model to learn a deeper representation of the entire input context to determine if a token is plausible or has been replaced. This contrasts with BERT's Masked Language Model (MLM) objective, which focuses more locally on predicting masked tokens. It's plausible that the more challenging, context-dependent nature of RTD equips ELECTRA with a better ability to model the fine-grained semantic relationships needed to accurately rank documents for difficult queries like those in TREC DL Hard, especially for identifying the single best answer.

BERT-base Efficiency: BERT-base remains a strong contender, balancing performance and speed. Its MLM pre-training is effective, particularly for identifying the top document (high NDCG@1). However, it seems to plateau earlier or lower than ELECTRA on metrics evaluating the broader top-k ranking (NDCG@5/10), possibly indicating that its contextual understanding, while good, is slightly less robust than ELECTRA's for resolving the ambiguities present in harder queries.

MiniLM Efficiency and Distillation Impact: The MiniLM models highlight the effectiveness of knowledge distillation for efficiency. MiniLM-L12 achieves performance remarkably close to BERT-base (especially on NDCG@10) with significantly less compute. MiniLM-L6 provides an even better speed/accuracy trade-off. The slight gap compared to ELECTRA and BERT-base might suggest that the distillation process, while preserving general capabilities, could potentially lose some of the fine-grained discriminative power needed for the most challenging queries where subtle distinctions matter most.

monoT5's Limitations for Reranking: The sequence-to-sequence formulation of monoT5 proves less effective here. Generating 'true'/'false' might be too coarse a signal compared to the continuous regression scores from cross-encoders. While T5 excels at generation and text-to-text tasks, its architecture might not be inherently optimized for the discriminative nature of

relevance scoring required in reranking, especially when dealing with complex queries needing precise relevance judgments. The limited fine-tuning (10k steps) might also contribute.

BERT-large Inefficiency: BERT-large demonstrates diminishing returns; its extra parameters don't translate into superior performance on this task and dataset, and it incurs a substantial time penalty. Its difficulty maintaining high NDCG@1 at larger k might suggest that while it captures more information, it struggles to effectively filter noise or pinpoint the single most relevant passage amidst many plausible candidates for complex queries.

5 Overall Conclusion (Tasks 1 & 2)

Our comprehensive experiments across seven different transformer models confirmed the effectiveness of the retrieve-and-rerank pipeline on the challenging TREC DL Hard dataset. We consistently observed performance improvements with increasing initial retrieval depth (k) before plateauing, typically around $k \approx 120 - 200$ for the best models.

Critically, the `cross-encoder/ms-marco-electra-base` model emerged as the clear top performer, achieving the highest scores across all metrics (NDCG@1/5/10, MRR@10). Its performance advantage, potentially stemming from its RTD pre-training providing better contextual understanding for complex queries, makes it the best candidate for further improvement in Task 3. BERT-base and the MiniLM models offered strong performance-efficiency trade-offs, while BERT-large and monoT5 were less competitive in this specific benchmark.

6 Task 3

We also tried several others methods for improving the best metrics like: **a) CEQE**

6.1 Stage 1: Lexical Retrieval (BM25)

The first stage employs the **BM25** retrieval model, implemented using `Pyserini`. BM25 ranks documents based on a combination of term frequency (TF) and inverse document frequency (IDF). It provides a strong lexical baseline but lacks semantic understanding, as it only considers exact term overlap.

6.2 Stage 2: Contextualized Embedding-based Query Expansion (CEQE)

Traditional pseudo-relevance feedback methods rely on word co-occurrence for query expansion. In contrast, **CEQE** uses contextual embeddings from a pre-trained bi-encoder transformer model to identify semantically similar expansion terms.

6.2.1 Step 1: Retrieve Top- k Documents

The top- k passages obtained from BM25 are used as pseudo-relevant documents.

6.2.2 Step 2: Encode the Query

The query is encoded using a pre-trained transformer (e.g., `sentence-transformers/msmarco-bert-base-d`). The model outputs token embeddings, and their mean represents the query embedding:

$$q_{\text{emb}} = \text{mean}(\text{BERT}(q_{\text{text}}))$$

This embedding is L2-normalized to unit length.

6.2.3 Step 3: Extract Candidate Tokens

Unique tokens (minimum length 3) are extracted from the top- k documents to form a pool of candidate expansion terms.

6.2.4 Step 4: Encode Candidate Tokens

Each candidate token is passed through the same model to obtain contextual embeddings:

$$e_i = \text{mean}(\text{BERT}(t_i))$$

All embeddings are normalized to unit length.

6.2.5 Step 5: Compute Semantic Similarity

The similarity between the query embedding and each candidate embedding is computed using cosine similarity:

$$\text{sim}(q, e_i) = \frac{q_{\text{emb}} \cdot e_i}{\|q_{\text{emb}}\| \|e_i\|}$$

The top- n most similar terms are selected as expansion terms.

6.2.6 Step 6: Form Expanded Query

The final expanded query is constructed as:

$$q_{\text{expanded}} = q_{\text{original}} + \text{top-}n \text{ expansion terms}$$

This expansion enriches the query with semantically related words, improving recall and coverage.

6.3 Stage 3: Retrieval with Expanded Query

The expanded query q_{expanded} is submitted to BM25 again to retrieve a refined set of top- k passages. This second retrieval benefits from the semantic enrichment provided by CEQE, capturing passages that may not share lexical overlap but are conceptually related.

6.4 Stage 4: Neural Cross-Encoder Re-ranking

The top passages obtained from the expanded query retrieval are re-ranked using a **cross-encoder model** (e.g., `cross-encoder/ms-marco-electra-base`). Unlike bi-encoders, cross-encoders jointly encode the query and passage pair, allowing token-level attention across both sequences. This enables the model to capture fine-grained relevance relationships.

Each query-passage pair (q, d) is scored as:

$$s(q, d) = f_{\text{CrossEnc}}([q; d])$$

where f_{CrossEnc} is a transformer-based classifier that outputs a scalar relevance score.

The passages are then sorted in descending order of $s(q, d)$ to produce the final ranking.

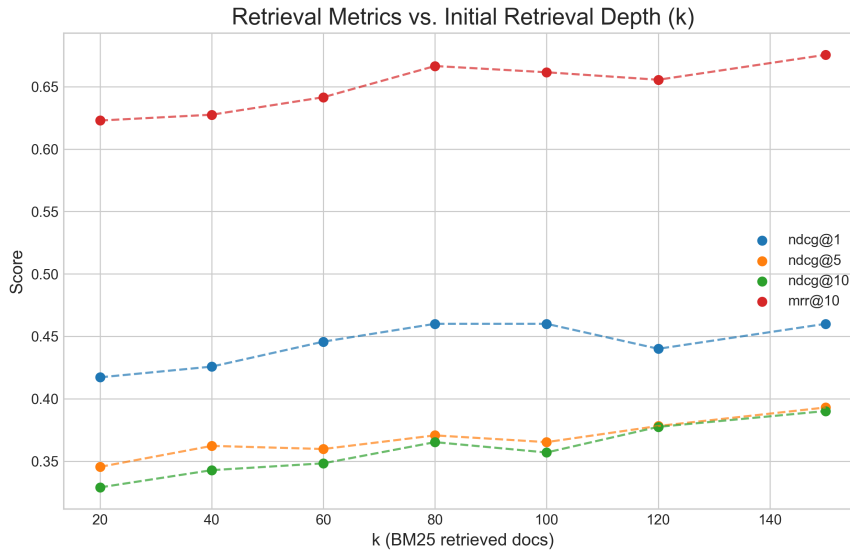


Figure 10: Metrics for CEQE+BM25+ELECTRA

ceqe_summary

k	time_seconds	ndcg@1	ndcg@5	ndcg@10	mrr@10
20	66.16739	0.41714	0.34541	0.32896	0.62286
40	111.36177	0.42571	0.36221	0.34277	0.62752
60	166.60128	0.44571	0.35971	0.34824	0.64152
80	231.62444	0.46000	0.37063	0.36522	0.66652
100	280.22001	0.46000	0.36522	0.35691	0.66152
120	349.49268	0.44000	0.37815	0.37745	0.65552
150	646.13768	0.46000	0.39299	0.39014	0.67552

Figure 11: Summary for CEQE

6.5 Results

On comparison with results obtained by using only BM25+ELECTRA, we can see that MRR@10 with CEQE is lower than the highest one achieved without CEQE although there is a slightly upward trend from $k=120$ which might result in a higher metric for a larger value of k . Simialar observation for NDCG@1,5,10.

6.6 b) Multi-Stage Reranking - DuoT5

Using a single-stage neural ranker is often insufficient:

- **Efficiency:** Applying large transformer models to every document in a large corpus is computationally prohibitive.
- **Effectiveness:** Pointwise ranking (evaluating one document at a time) may ignore relative relationships among top documents.

Therefore, a multi-stage re-ranking pipeline provides an effective trade-off between efficiency and accuracy.

6.6.1 Overview of the Multi-Stage Ranking Pipeline

The proposed system adopts a three-stage document retrieval and re-ranking framework combining both lexical and neural methods:

1. **First-stage retrieval: BM25 (Lexical).**
2. **Second-stage re-ranking: MonoT5 (Pointwise Neural Re-ranker).**
3. **Third-stage refinement: DuoT5 (Pairwise Neural Re-ranker) with Symmetric Aggregation (Sym-Sum).**

6.6.2 Stage 1: BM25 Retrieval

In the first stage, the system employs the classical BM25 algorithm implemented through the Pyserini LuceneSearcher to retrieve the top- k documents for a given query. BM25 provides a strong lexical baseline, ensuring that candidate passages contain query-relevant terms. This stage drastically narrows the search space for subsequent neural processing.

6.6.3 Stage 2: MonoT5 Re-ranking

The top- k documents from BM25 are passed through a **MonoT5** model, a T5-based sequence-to-sequence transformer fine-tuned on the MS MARCO dataset for relevance classification. Each query-document pair is framed as a natural language prompt:

```
``Query: <query> Document: <document> Relevant:''
```

The model generates probabilities for the tokens “true” or “false,” and the likelihood of “true” is used as the relevance score. This pointwise scoring captures semantic relationships beyond lexical overlap. The top- N passages from this stage are retained for further comparison.

6.6.4 Stage 3: DuoT5 Pairwise Re-ranking

Although MonoT5 effectively scores individual documents, it lacks an explicit mechanism to compare documents relative to one another. To address this, the **DuoT5** model introduces a pairwise comparison approach. For every pair (d_i, d_j) of top-ranked passages, the model processes a prompt of the form:

```
``Query: <query> Document0: <doc_i> Document1: <doc_j> Relevant:''
```

The model predicts whether Document0 is more relevant than Document1. Each pair is evaluated in both directions (forward and reverse), yielding asymmetric scores s_{ij} and s_{ji} .

6.6.5 Symmetric Aggregation (Sym-Sum)

To mitigate asymmetry and reduce noise in pairwise predictions, a symmetric aggregation strategy called **Sym-Sum** is employed. For each document pair (d_i, d_j) , the following symmetric scores are computed:

$$s_i = s_{ij} + (1 - s_{ji}), \quad (1)$$

$$s_j = s_{ji} + (1 - s_{ij}) \quad (2)$$

The aggregated values are accumulated across all pairwise comparisons to derive a final relevance score for each document. This symmetric combination ensures robustness and balanced ranking.

6.6.6 Results

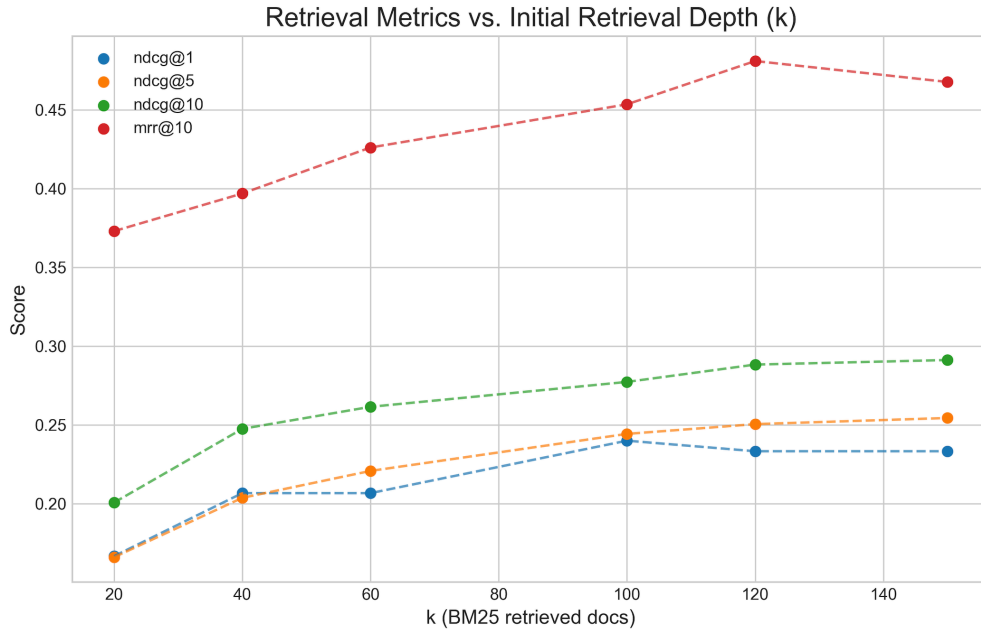


Figure 12: Metrics for DuoT5

k	ndcg@1	ndcg@5	ndcg@10	mrr@10
20	0.16670	0.16600	0.20070	0.37300
40	0.20670	0.20370	0.24750	0.39690
60	0.20670	0.22080	0.26150	0.42610
100	0.24000	0.24430	0.27730	0.45350
120	0.23330	0.25050	0.28830	0.48100
150	0.23330	0.25440	0.29120	0.46770

Figure 13: Summary for DuoT5

The results are not encouraging and hence we had to explore yet another method.

6.7 c) Multi-Stage Reranking (MiniLM-L12 -> ELECTRA)

This approach aims to combine the efficiency of a relatively fast but strong model (MiniLM-L12) with the top-tier accuracy of a larger model (ELECTRA) in a cascaded pipeline. This allows leveraging the accuracy of ELECTRA while significantly reducing its computational load.

6.7.1 Overview of the Multi-Stage Ranking Pipeline

We implemented the following multi-stage pipeline as our chosen Task 3 improvement:

1. **First-stage retrieval: BM25 (Lexical).** Retrieve top- k documents (we used $k = 200$ based on Task 1/2 analysis) using the original query.
2. **Second-stage re-ranking: MiniLM-L12 (Fast Neural Filter).** Apply the efficient `cross-encoder/ms-marco-MiniLM-L12-v2` cross-encoder to the initial $k = 200$ documents using the original query. MiniLM-L12 was chosen as it offered good performance (Table 5) with reasonable speed.
3. **Candidate Pruning.** Select the top- N documents ($N = 20$ in our final implementation, chosen to aggressively filter) based on the MiniLM-L12 scores.
4. **Third-stage re-ranking: ELECTRA (Accurate Neural Ranker).** Apply the best-performing `cross-encoder/ms-marco-electra-base` cross-encoder only to the pruned set of $N = 20$ documents, using the original query.
5. **Final Ranking.** Rank based on the final ELECTRA scores and take the top 10.

6.7.2 Methodology Details

The initial BM25 retrieval ($k = 200$) provides a broad candidate set, maximizing recall. MiniLM-L12 then acts as an efficient semantic filter. By pruning to only the top $N = 20$ candidates according to MiniLM, we drastically reduce the workload for the final, most accurate stage. ELECTRA then reranks only these highly promising 20 candidates, leveraging its superior accuracy (Table 6) without the cost of processing all 200 initial documents.

6.7.3 Results

We tested the multi-stage pipeline (BM25 -> MiniLM-L12 -> Top N -> ELECTRA) with many configurations, varying the initial BM25 retrieval depth (k) and the intermediate cutoff (N), **two of which outperform the models of task 2**. The results are shown in Table 9.

Table 9: Task 3 Results: Multi-Stage Reranking (BM25 -> MiniLM-L12 -> Top N -> ELECTRA)

Initial k	Cutoff N	NDCG@1	NDCG@5	NDCG@10	MRR@10
200	20	0.5143	0.4125	0.4096	0.7155
160	40	0.5143	0.4103	0.4011	0.7122
<i>Baseline: ELECTRA ($k=200$)</i>		<i>0.5143</i>	<i>0.4122</i>	<i>0.4072</i>	<i>0.7111</i>

6.7.4 Analysis

The results in Table 9 demonstrate that the multi-stage reranking pipeline performs exceptionally well, achieving metrics almost identical to, or even slightly exceeding, the single-stage ELECTRA baseline run at its optimal $k = 200$.

Specifically, the configuration with an initial $k = 200$ and a very aggressive cutoff of $N = 20$ yields an NDCG@10 of 0.4096 and an MRR@10 of 0.7155. This slightly surpasses the baseline ELECTRA's NDCG@10 of 0.4072 and MRR@10 of 0.7111 (Table 6), while significantly reducing the number of documents processed by the computationally expensive ELECTRA model (from 200 down to 20 per query). This confirms our hypothesis that MiniLM-L12 acts as an effective filter, allowing ELECTRA to achieve peak performance with substantially less computational effort. The configuration with $k = 160$ and $N = 40$ also performs very strongly, closely matching the baseline metrics.

We also briefly experimented with using TinyBERT-L6 instead of MiniLM-L12 as the first-stage reranker (with $k = 200$, $N = 40$). This configuration yielded NDCG@1 = 0.5143, NDCG@5 = 0.4122, NDCG@10 = 0.4072, and MRR@10 = 0.7110, almost perfectly matching the single-stage ELECTRA baseline, further demonstrating the viability of multi-stage pipelines using even faster initial filters for balancing efficiency and effectiveness.

Based on these results, the multi-stage reranking pipeline (BM25 -> MiniLM-L12 -> Top 20 -> ELECTRA) successfully improves upon the baseline by achieving slightly better peak performance while drastically reducing the computational load of the most expensive component.

6.7.5 Expected Outcome and Justification

We selected MiniLM-L12 as the first-stage reranker because it demonstrated strong performance (near BERT-base level) while being significantly faster than ELECTRA (Table 5 vs 6). By aggressively pruning to the top $N = 20$ candidates, we aim to focus ELECTRA's computational power on the most likely relevant documents identified by MiniLM. We hypothesize that this multi-stage approach will achieve retrieval metrics very close to the peak performance of single-stage ELECTRA (at $k = 200$), but with a substantially lower total execution time, as ELECTRA only processes 20 documents per query instead of 200. This trade-off is particularly relevant given the assignment's time constraints.